

Risc-V Multi-Cycle 설계 & UART_Peripheral 검증

발표자 : 조민준

프로젝트 개요

RV32I 기반의 Multi-Cycle CPU 설계

APB + Peripheral 제작 및 CPU와 연동

SystemVerilog 를 이용해 Testbench 제작 후(class사용) UART 기능 검증

C언어 코드를 컴파일 후 기계어를 ROM에 로드



RISC-V Multi-Cycle

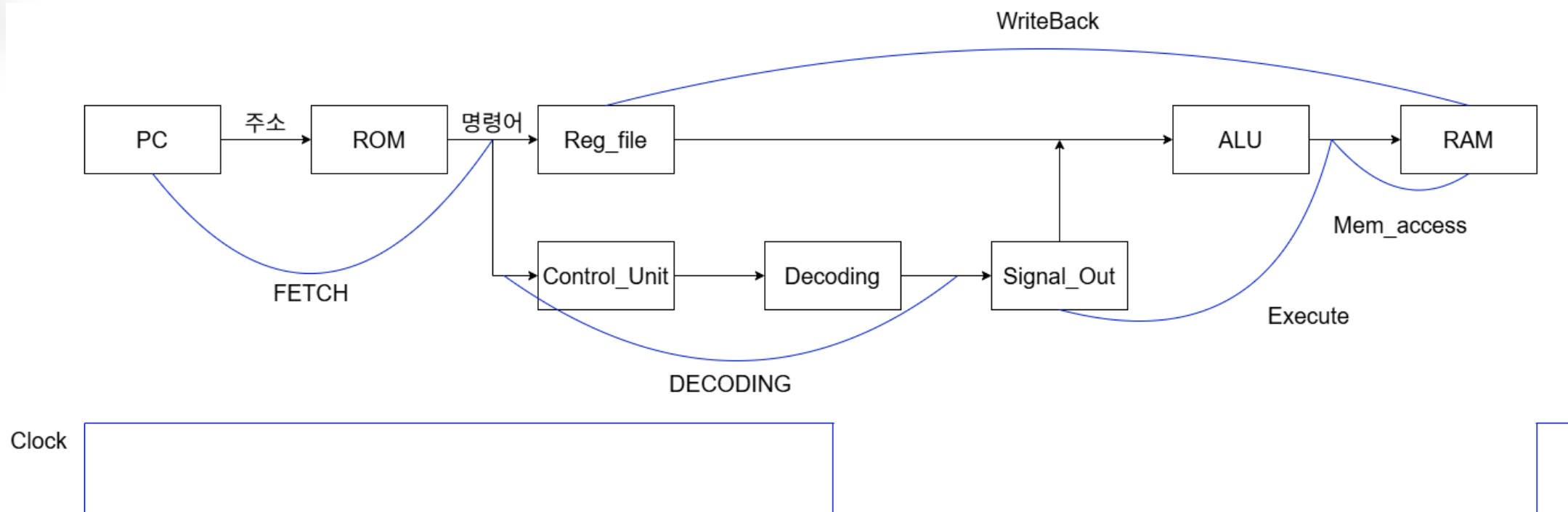
Single-Cycle CPU

- 특징 : Single-Cycle에서는 모든 명령어를 한 사이클(1CLK)에 처리
- 단점 : 1CLK의 주기를 가장 실행 시간이 긴 타입에 맞춰서 설계해야 하기에 클럭 효율이 낮음(클럭 주기가 길다)

Multi-Cycle CPU

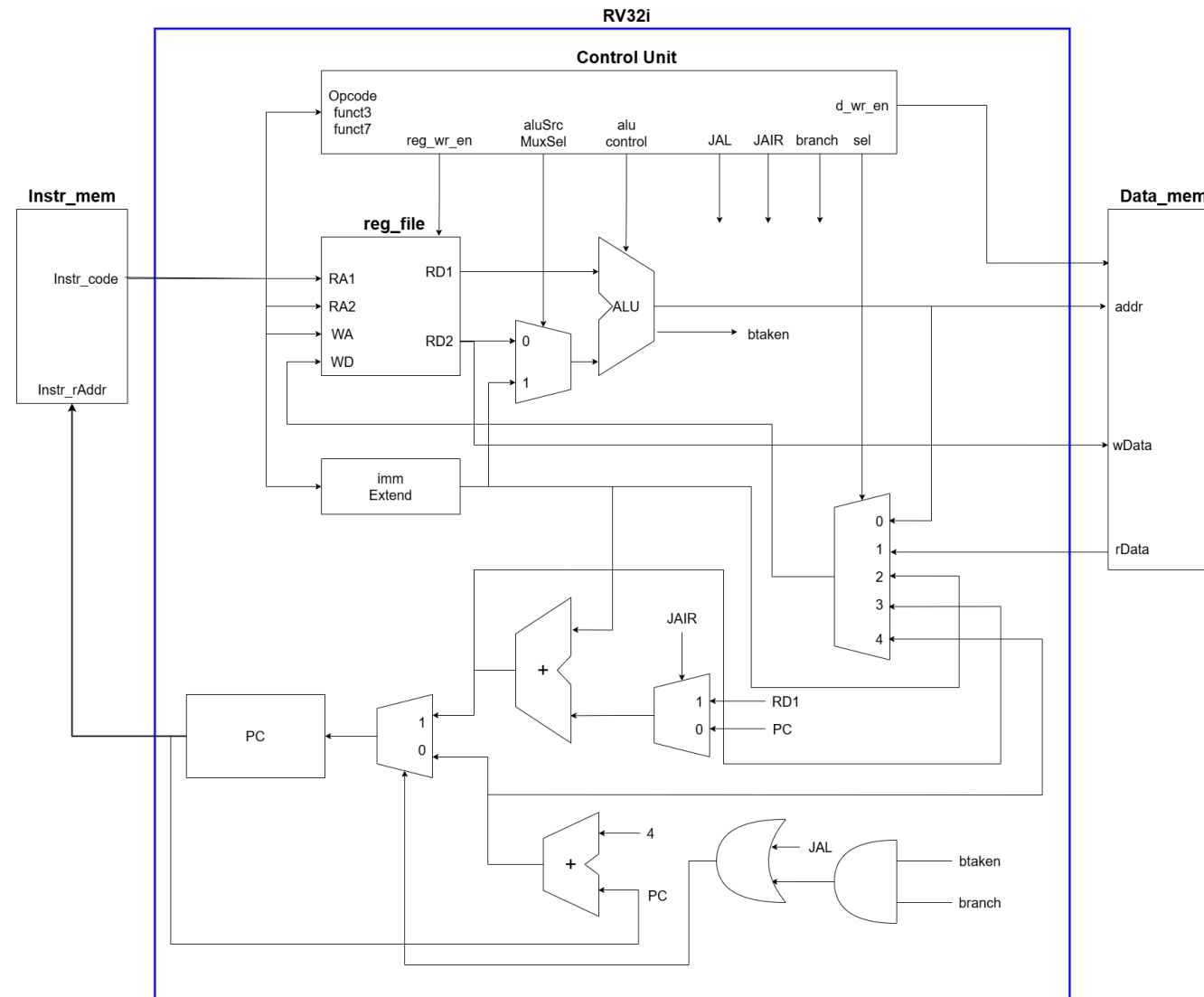
- 특징 : 명령어의 수행 과정을 여러 단계로 나누어 처리
- 장점 : 명령어 종류에 따라 필요한 단계만 수행하기에 클럭 효율이 높음(클럭 주기를 짧게 만들 수 있음)

RISC-V Multi-Cycle

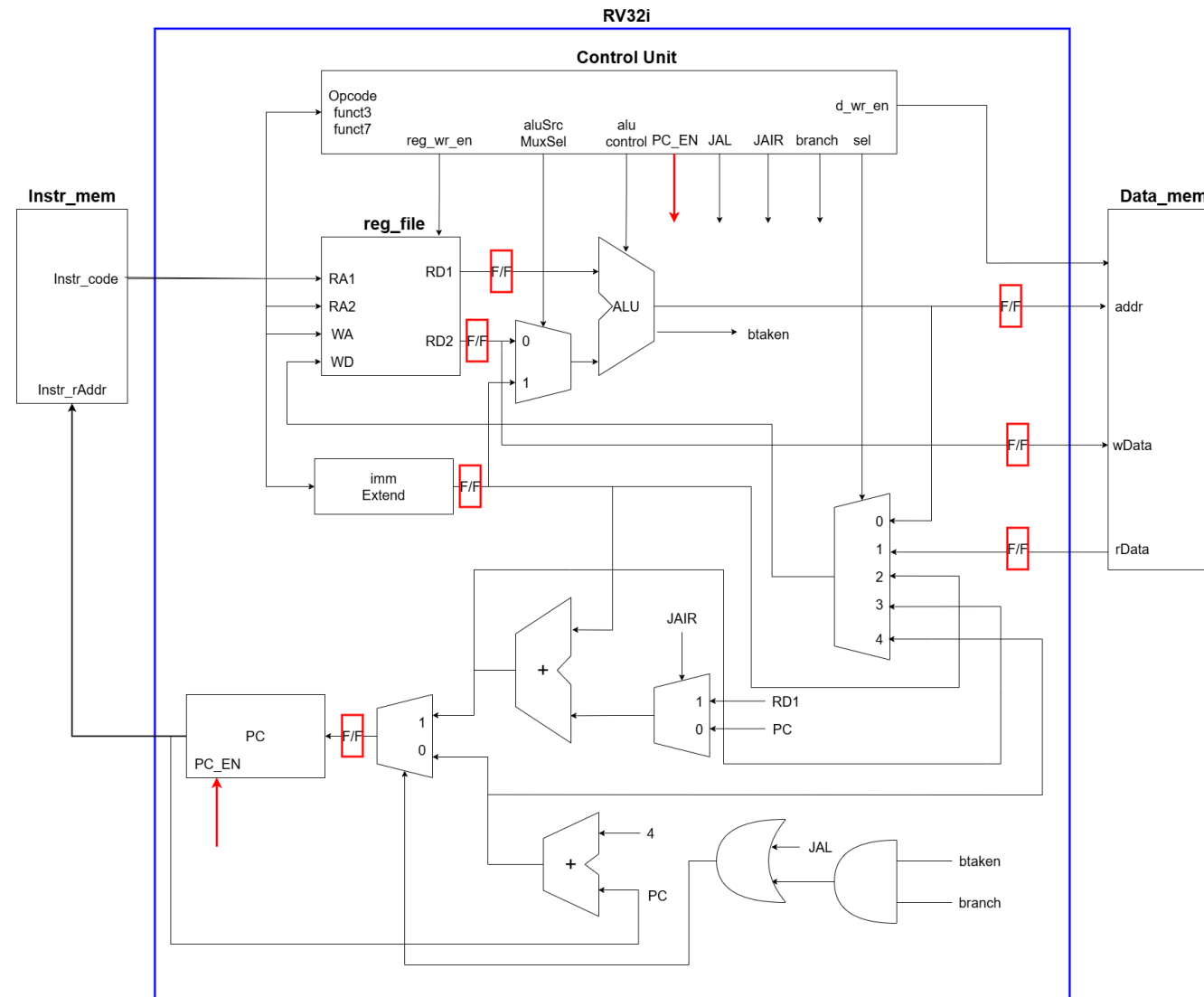


그림과 같이 명령어의 수행 과정을 여러 단계로 나누어 필요한 단계만 진행하도록 설계

RISC-V Single-Cycle Diagram



RISC-V Multi-Cycle Diagram



AMBA BUS

BUS

- 칩이나 시스템 내부에서 여러 장치가 데이터를 주고받을 수 있게 해주는 통로

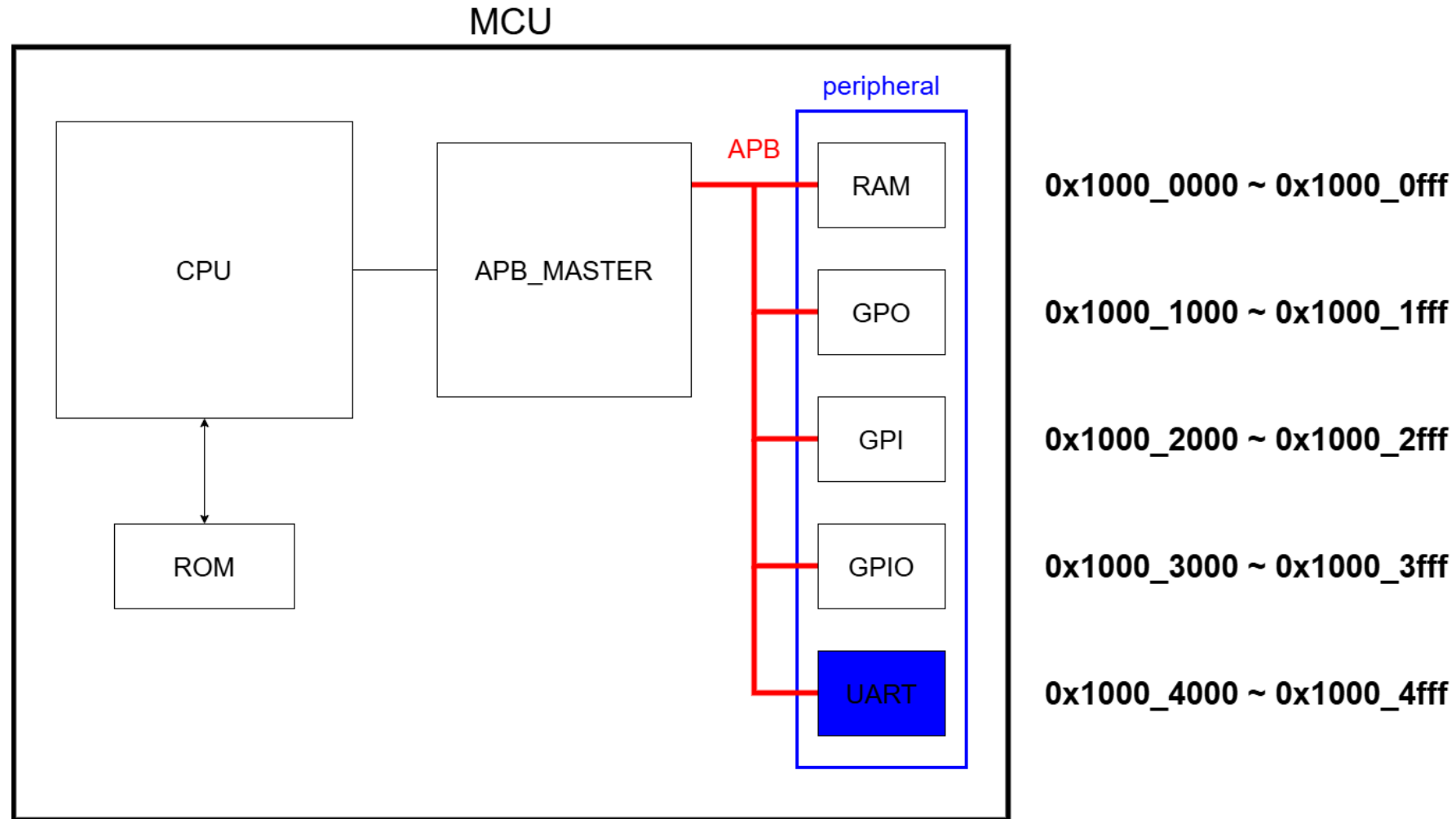
AMBA(Advanced Microcontroller Bus Architecture)

- ARM사가 개발한 버스 표준으로 칩 내의 여러 하드웨어 모듈을 효율적으로 연결하고 관리하기 위한 규격

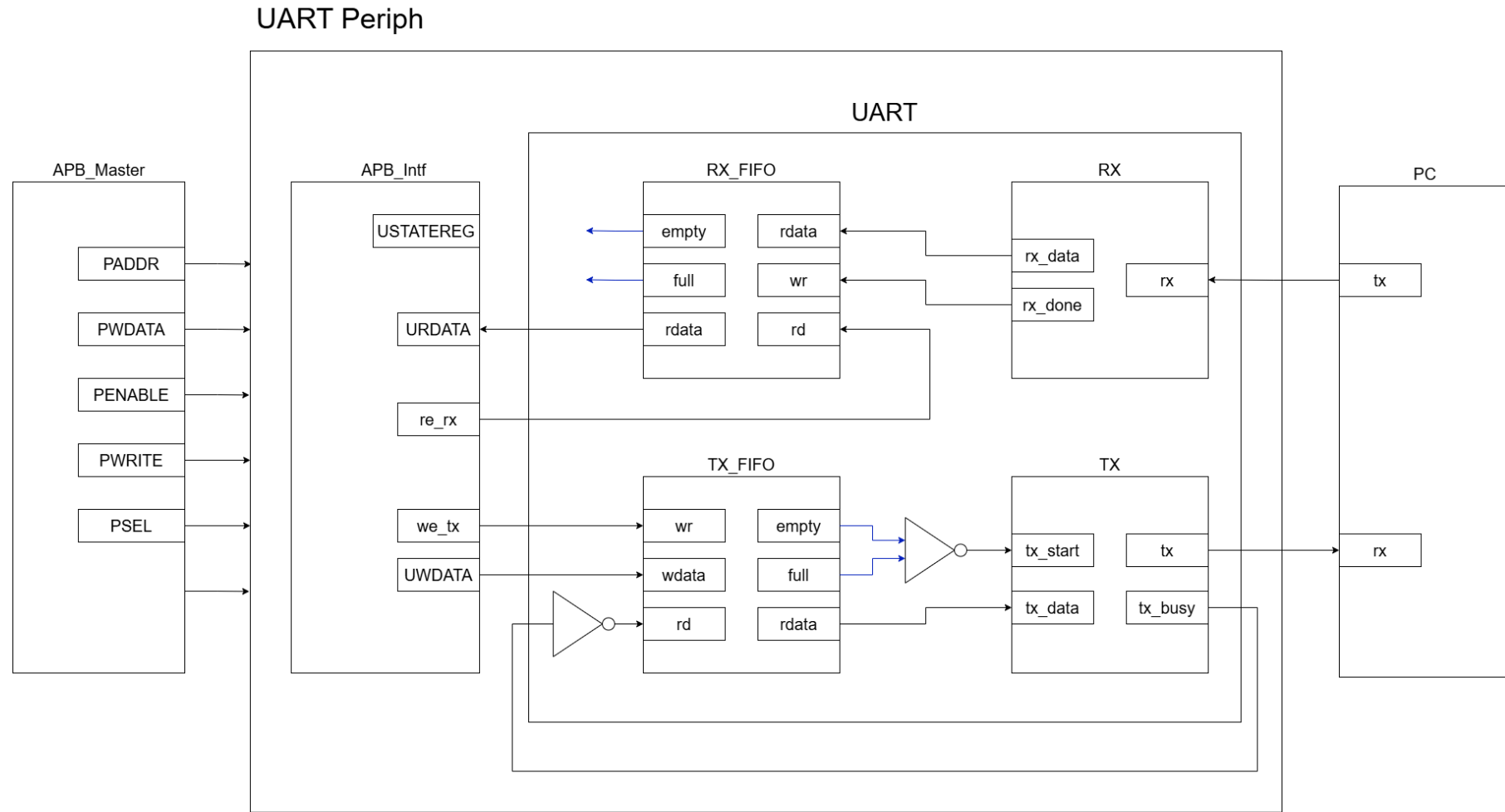
APB(Advanced Peripheral Bus)

- AMBA Protocol의 일부로, Peripheral제어에 특화된 저전력, 저비용 버스. 주로 peripheral의 register를 제어하는 용도로 사용

AMBA BUS Diagram



UART 설계



파란 선은 USTATereg로 들어감

Verification

검증 방식

TX : 랜덤으로 넣은 rdata값이 reg2[tx write용 reg] 에 잘 적히는지 확인

RX : 랜덤으로 reg3[rx read용 reg] 에 넣은 값이 PRDATA로 잘 읽히는지 확인

Verification Code

```
class transaction;
    logic        transfer;
    logic        write;
    logic [31:0] tx;
    logic [31:0] rx;
    rand logic [31:0] addr;
    rand logic [31:0] wdata;
    logic [31:0] rdata;
    constraint c_addr {
        addr inside {
            [32'h1000_4000 : 32'h1000_400F]
        }
    }
endclass
```

Addr과 wdata는 랜덤으로 입력,
주소지는 UART에 해당하는 주소만 사용하도록 제한

```
task automatic run(int loop);
    int i;
    for (i = 1; i <= loop; i++) begin
        t.randomize();
        t.addr = 32'h1000_4008;
        tx();
        rx(t.wdata[7:0]);
    end
endtask
```

tx 검증을 위해 addr 고정
넣은 loop값 만큼 반복 진행

rx로 t.wdata 입력

Verification Code

```
task automatic tx();
    t.transfer = 1'b1;
    t.write    = 1'b1;

    m_if.transfer <= t.transfer;
    m_if.write    <= t.write;
    m_if.addr     <= t.addr;
    m_if.wdata    <= t.wdata;
    @(posedge m_if.clk); //
    //(transfer=1 상태 유지)
    @(posedge m_if.clk);
    // PREADY 대기
    wait (m_if.ready);
    // APB Write 종료 초기화
    m_if.transfer <= 1'b0;
    m_if.write    <= 1'b0;
    m_if.addr     <= '0;
    m_if.wdata    <= '0;
    t.print("TX_APB_END");
    # (TIME_FOR_UART_BIT * 10);
```

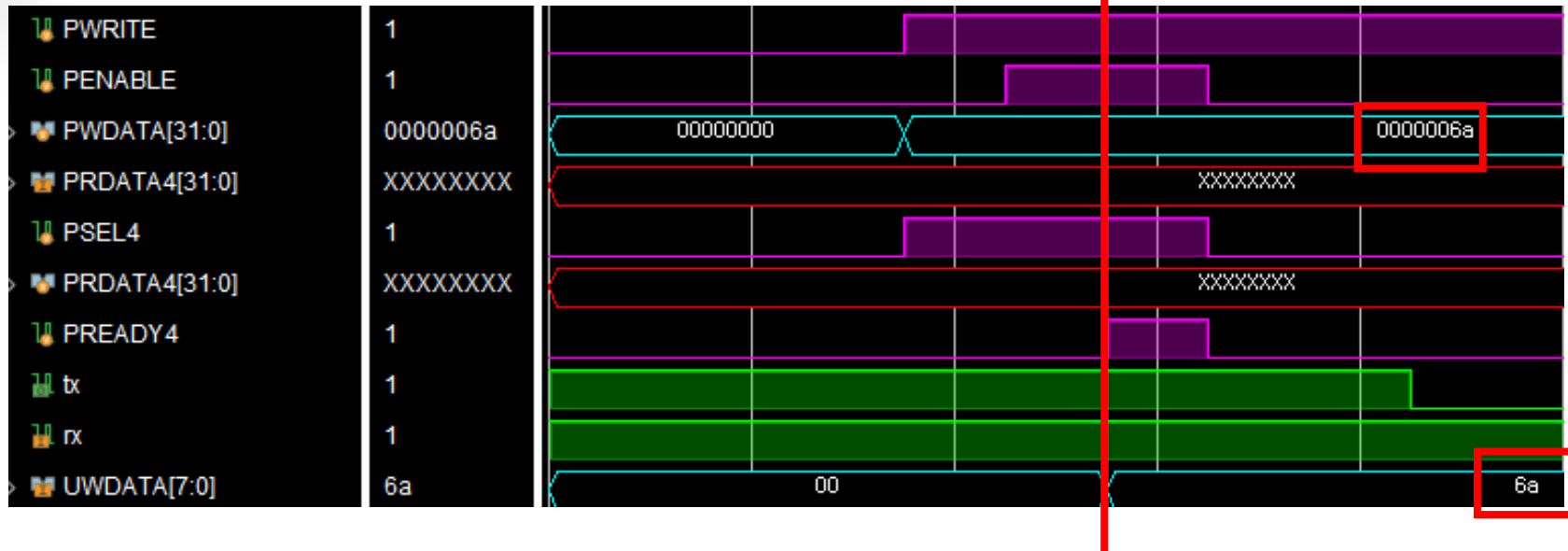
```
task automatic rx(input logic [7:0] data_to_send);
    int i;
    m_if.rx <= 1'b0;
    #TIME_FOR_UART_BIT;

    for (i = 0; i < 8; i++) begin
        m_if.rx <= data_to_send[i];
        #TIME_FOR_UART_BIT;
    end

    m_if.rx <= 1'b1;
    #TIME_FOR_UART_BIT;
    #TIME_FOR_UART_BIT;

    // 2. APB Read Transaction 시작
    t.transfer = 1'b1;
    t.write    = 1'b0;
    t.addr     = 32'h1000_400C;
```

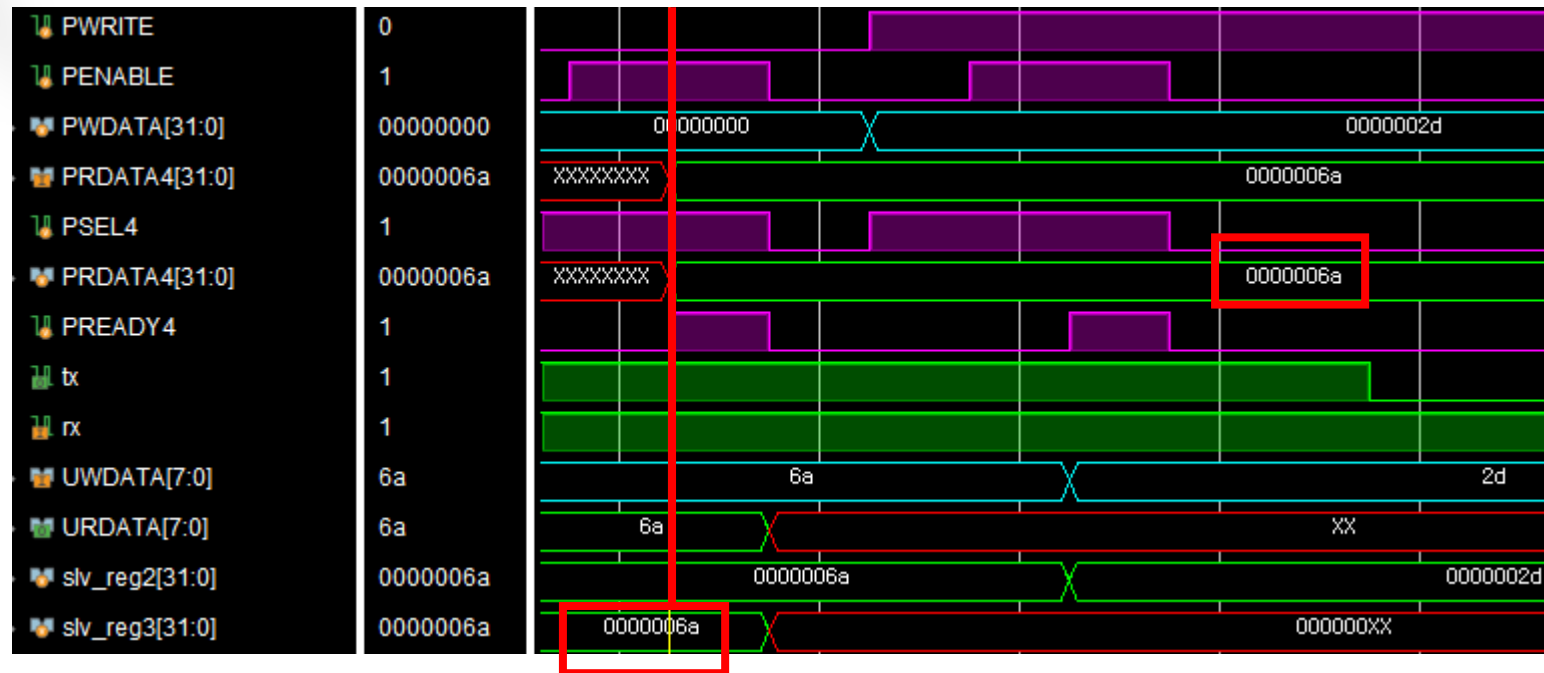
TX Verification



PWRITE = 1, PENABLE = 1, PSEL4 = 1, PREADY4 = 1

PWDATA에 있는 6a가 UWDATA로 잘 들어가는 모습을 확인 가능

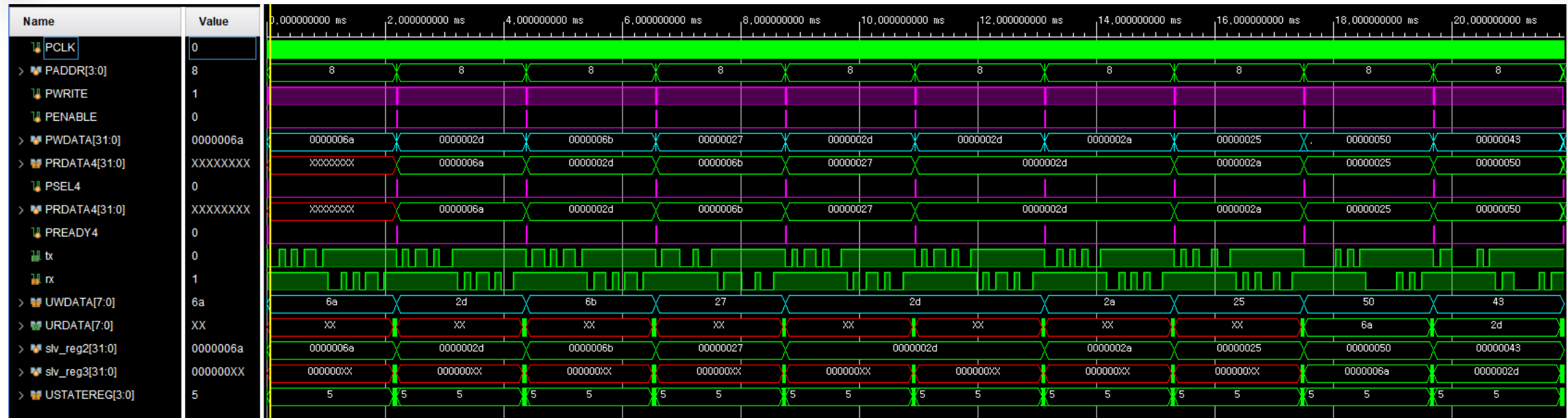
RX Verification



PENABLE = 1, PSEL4 = 1, PREADY4 = 1

reg3에 있는 6a가 PRDATA4로 잘 들어간 모습을 확인 가능

Simulation Wave



C_Code

```
#define APB_BASE_ADDR 0x10000000

#define GPO_OFFSET 0x1000
#define GPI_OFFSET 0x2000
#define GPIO_OFFSET 0x3000
#define UART_OFFSET 0x4000

#define GPO_BASE_ADDR (APB_BASE_ADDR + GPO_OFFSET)
#define GPI_BASE_ADDR (APB_BASE_ADDR + GPI_OFFSET)
#define GPIO_BASE_ADDR (APB_BASE_ADDR + GPIO_OFFSET)
#define UART_BASE_ADDR (APB_BASE_ADDR + UART_OFFSET)

#define GPO_CR (*(uint32_t*)(GPO_BASE_ADDR + 0x00))
#define GPO_ODR (*(uint32_t*)(GPO_BASE_ADDR + 0x04))

#define GPI_CR (*(uint32_t*)(GPI_BASE_ADDR + 0x00))
#define GPI_IDR (*(uint32_t*)(GPI_BASE_ADDR + 0x04))

#define GPIO_CR (*(uint32_t*)(GPIO_BASE_ADDR + 0x00))
#define GPIO_ODR (*(uint32_t*)(GPIO_BASE_ADDR + 0x04))
#define GPIO_IDR (*(uint32_t*)(GPIO_BASE_ADDR + 0x08))

#define UART_USR (*(uint32_t*)(UART_BASE_ADDR + 0x00))
#define UART_UWD (*(uint32_t*)(UART_BASE_ADDR + 0x08))
#define UART_URD (*(uint32_t*)(UART_BASE_ADDR + 0x0c))
```

레지스터의 주소를 정의, 간단하게 접근할수 있도록
만들

C_Code

```
if (uart_char_available()) {
    rx_char = (char)UART_URD;
    switch (rx_char) {
        case 'l':
            if (ledState != LEFT) {
                ledState = LEFT;
                ledData = 0x01;
            }
            break;

        case 'r':
            if (ledState != RIGHT) {
                ledState = RIGHT;
                ledData = 0x80;
            }
            break;
    }
}

if (ledState == LEFT) {
    LED_leftShift(&ledData);
} else {
    LED_rightShift(&ledData);
}

LED_write(ledData);
delay(200);
time_accumulator += 200;
```

UART로 문자를 받으면 즉시 읽어서 방향 전환
l입력시 LED가 왼쪽으로, r 입력시 LED가 오른쪽으
로 쉬프트

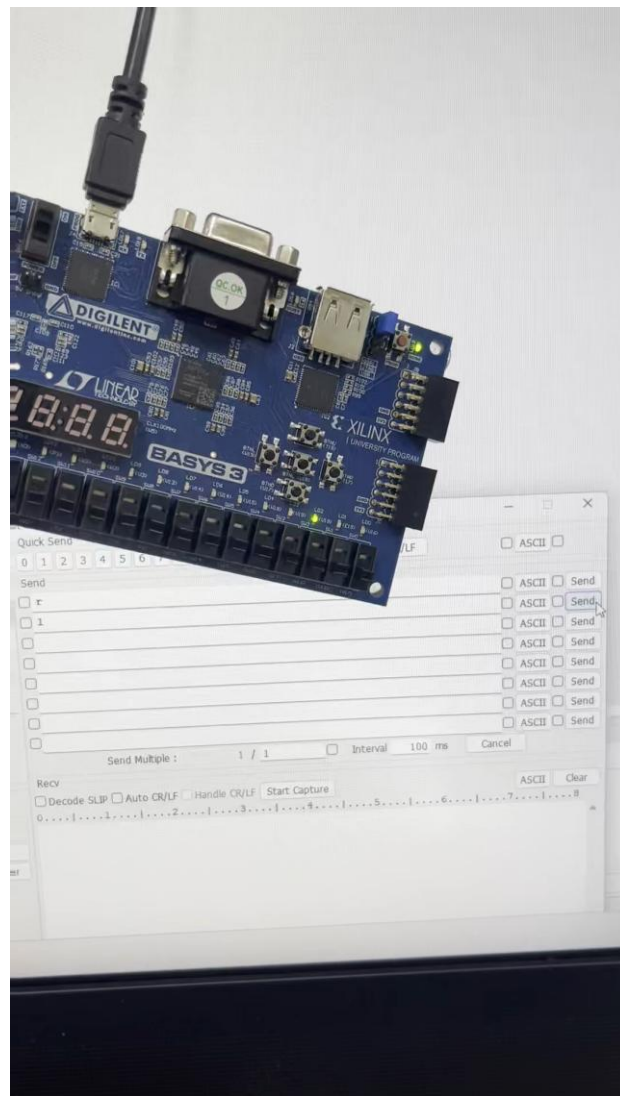
C_Code

```
switch (rx_char) {  
    case 'l':  
        if (ledState != LEFT) {  
            ledState = LEFT;  
            ledData = 0x01;  
            uart_put_char('s');  
            uart_put_char('e');  
            uart_put_char('n');  
            uart_put_char('d');  
            uart_put_char(' ');  
            uart_put_char('l');  
            uart_put_char('e');  
            uart_put_char('f');  
            uart_put_char('t');  
            uart_put_char(' ');  
            uart_put_char('s');  
            uart_put_char('h');  
            uart_put_char('i');  
            uart_put_char('f');  
            uart_put_char('t');  
            uart_put_char('\n');  
        }  
        break;
```

“L”버튼 누를 시 -> send left shift 전송
5초마다 left shift라는 상태 전송
스탑과 R버튼 누를시에도 동일

```
if (time_accumulator >= REPORT_INTERVAL) {  
    if (ledState == LEFT) {  
        uart_put_char('l');  
        uart_put_char('e');  
        uart_put_char('f');  
        uart_put_char('t');  
        uart_put_char(' ');  
        uart_put_char('s');  
        uart_put_char('h');  
        uart_put_char('i');  
        uart_put_char('f');  
        uart_put_char('t');  
        uart_put_char('\n');
```

동작 영상



고찰

BUS, Peripheral에 대해 이해함.

C언어를 통해서 하드웨어를 직접 작동시키는 과정을 통해서 하드웨어의 작동방식에 대한 이해도를 높힐 수 있었음